

SDA AI DATA CENTER BOOTCAMP · AUG 2026

Inference is a memory game

Week 3 · Day 1 · GPU Systems & AI Operations



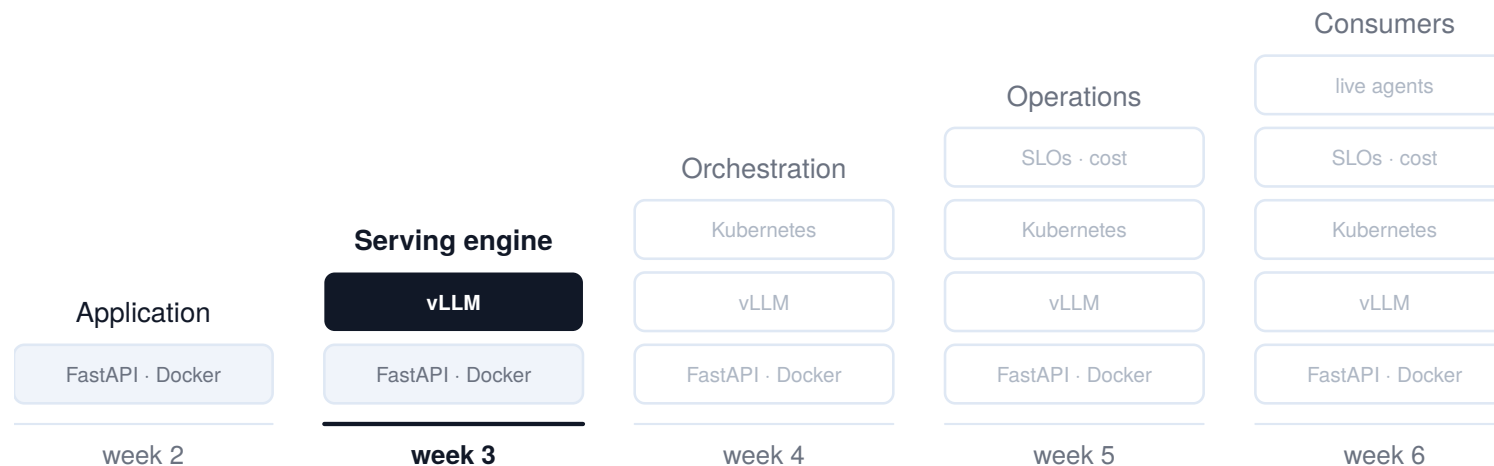
Three from last week

- What did the OpenAI `/v1` contract let a consumer ignore about your service?
- What is the difference between a Docker image and a running container?
- Compose arrived on Thursday. What did it add over a single `docker run` ?

EVERY SUNDAY

The map

One system. A layer a week, and the stack is the course.



- Week 3 lights up. The serving engine is this week's layer.
- Everything above it stays where it was. The endpoint contract does not move.
- This week the engine changes and nothing above it does.



THE THREAD

Memory before speed

Every optimisation this week starts by asking where the bytes are and how far they travel.



Why memory, not FLOPs

- The card does far more math per second than it can move numbers per second.
- Serving one user, the cores spend most of their time waiting on memory.
- So the number that sets speed is usually bytes moved, not math done.
- Today: the hardware that makes this true. The rest of the week spends it.



The GPU as an operator sees it

You do not need the physics. You need the shape.



A throughput machine, not a fast CPU

- A CPU has a few large cores tuned to finish one task quickly.
- A GPU has thousands of small cores, grouped into units called SMs (streaming multiprocessors - the T4 has 40).
- Each core is slower than a CPU core. There are just thousands of them.
- It wins by doing the same operation across huge batches of numbers at once.

A GPU is built for throughput. It is fast only when you keep all those cores fed with work.

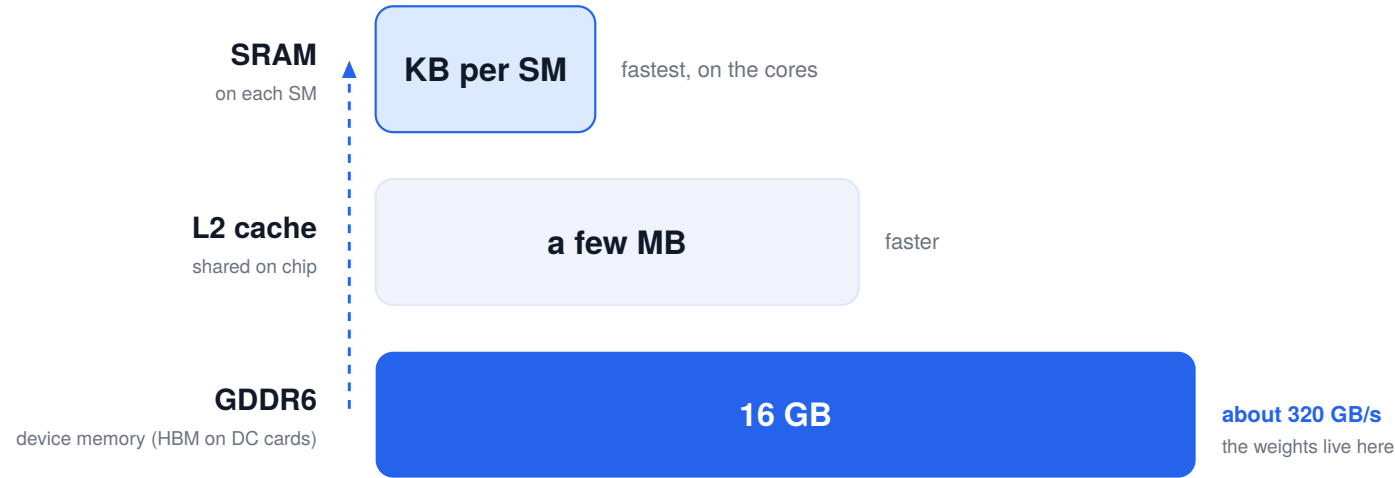
The memory hierarchy

Where the weights live, and where they must travel every token.



Three tiers on a T4

Smaller and faster as you climb. Bigger and slower as you descend.



- Device memory: 16 GB of GDDR6 at about 320 GB/s. The model weights live here.
- L2 cache: a few MB, shared across the chip, much faster.
- SRAM: kilobytes per SM, sitting on the cores, faster again.

The trip is the cost

- To do math on a weight, the core has to pull it up from device memory.
- Device memory is the slow tier (slow only against the caches), and the weights are large.
- Decode reads every weight once per token generated.
- So the weights make the whole trip, over and over, one token at a time.

The weights do not fit in the fast memory. They live in device memory and commute to the cores. That commute sets the pace.

Bandwidth vs FLOPs

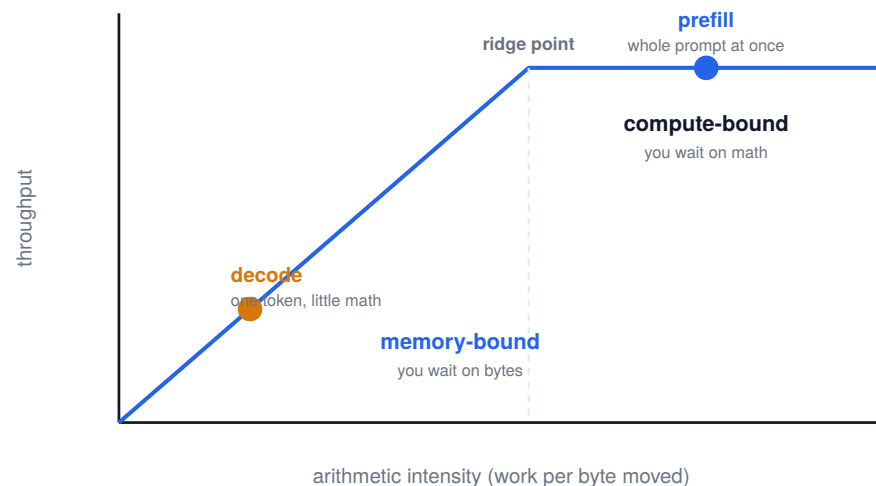
How much work do you get per byte you move?



Work per byte moved

- The card has two separate budgets: math per second, and bytes per second.
- Arithmetic intensity is the ratio: how much math you do per byte you move.
- Low intensity means you move a lot and compute a little. You wait on memory.
- High intensity means you reuse each byte for lots of math. You wait on cores.

The roofline, as intuition



- Prefill reads the whole prompt at once and reuses the weights: it sits right.
- Decode does little math per token but drags every weight past the cores: it sits left.

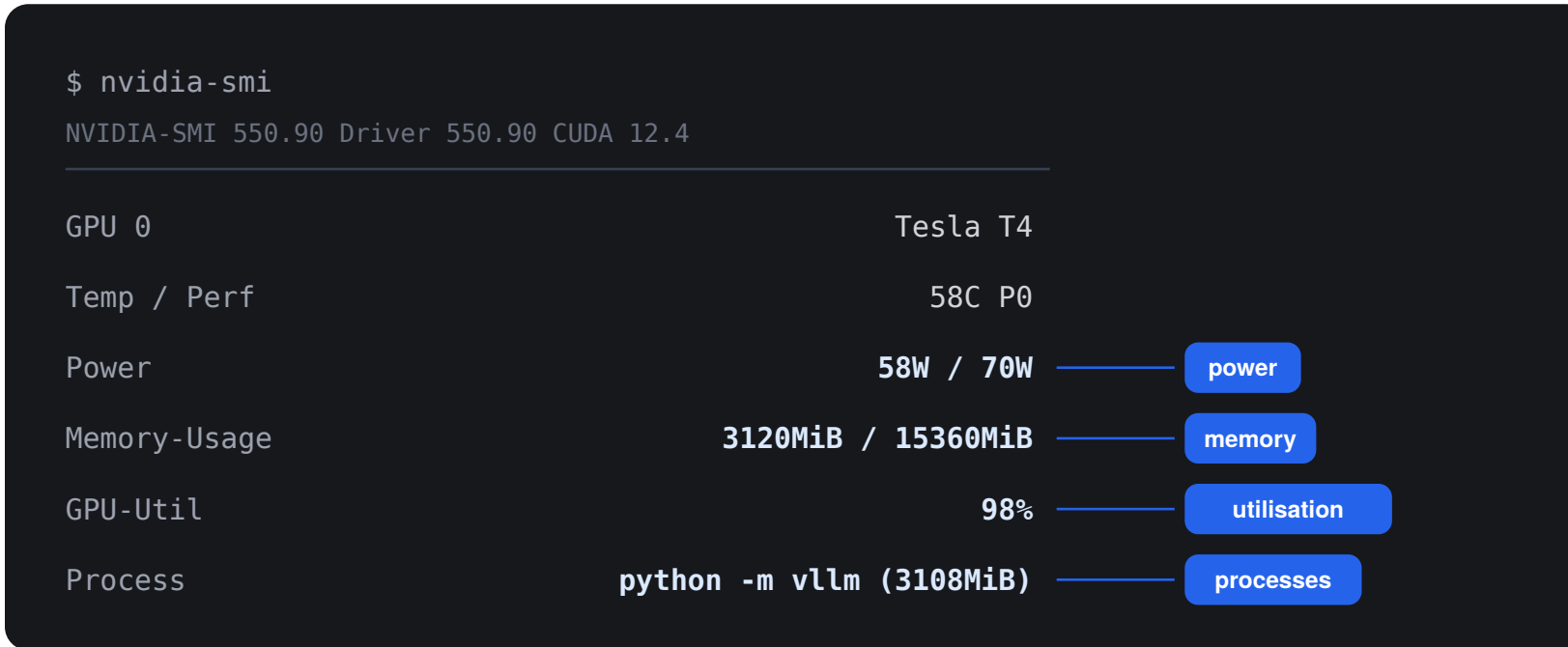
Reading nvidia-smi

The one tool you run first, every time, on every GPU box.

One real capture

```
$ nvidia-smi
NVIDIA-SMI 550.90 Driver 550.90 CUDA 12.4

GPU 0                               Tesla T4
Temp / Perf                         58C P0
Power                               58W / 70W
Memory-Usage                        3120MiB / 15360MiB
GPU-Util                            98%
Process                             python -m vllm (3108MiB)
```



- This is a T4 serving the lab model under a single request.
- Four fields carry almost everything you need day to day.
- We take them one at a time.



Memory-usage

- `3120MiB / 15360MiB` : used over total, on this card.
- The 3.1 GB is the model weights, the number you predicted in week 2.
- Total shows about 15 GB usable on a 16 GB T4. The rest is reserved.
- Watch this climb as context grows: that growth is the KV cache, Monday's topic.

Memory-usage is the field that tells you whether the next request fits. Learn to read it first.

Utilisation, and its trap

- GPU-Util 98% looks like the card is working hard. It is not what it sounds like.
- Utilisation means a kernel was running during the sample window - and this 'kernel' is not week 2's: there it was the operating system's core; here it is one small program the GPU runs. Nothing more is claimed.
- It does not mean the cores were full, or the memory bandwidth was saturated.
- The sampled % is 'was a kernel running', not 'was the card well used': the lab measures throughput multiplying under batching while util climbs far less.

Utilisation says a kernel ran. It does not say the GPU was used well. This is the trap.

Power, clocks, processes

- `Pwr 58W / 70W` : draw against the card's cap. A real signal of effort.
- Clocks: the running frequency. Drops under heat or power limits (thermal throttling).
- Processes: which PIDs hold memory on the card, and how much each holds.
- Power tracks true work better than utilisation: a full card pulls near its cap.

The fields that matter

Memory-usage

Does the next request fit? Where the KV cache shows up.

Utilisation

A kernel ran. Not that the card was used well.

Power

The honest signal of how hard the card is actually working.

DCGM, one slide

- nvidia-smi shows you one box, right now, by hand.
- DCGM is NVIDIA's fleet tool: the same numbers, sampled continuously, across many GPUs.
- It exposes utilisation, memory, power, temperature, and throttling as metrics you can scrape.
- Week 5 scrapes DCGM into Prometheus so the dashboards watch every card at once.

Same numbers, at fleet scale, over time. nvidia-smi is the hand tool; DCGM is the monitoring feed.

CHECK YOURSELF

Check yourself

- One small request on this machine: why is it a bad fit? Say it in terms of cores.
- Which memory tier holds the 3 GB of weights, and why that one?
- GPU-Util 98% during single-request decode. Is the card well used?



nvidia-smi under load

- Now the demo: the golden stack - a 7B AWQ on the instructor's 4090, your lab model's bigger cousin. Same anatomy, bigger numbers; your T4 runs the 1.5B.
- We run `nvidia-smi` in a loop while a request streams in.
- Watch memory-usage sit flat on the weights, then climb as the context grows.
- Watch one request's util reading, then remember it this afternoon when batching multiplies throughput without util following.

During the demo, keep the memory-usage and utilisation fields in view. This afternoon you produce both across a matrix.

Predict before you measure

Credited for handing it in. Never marked right or wrong. Write both numbers down before you open Colab. The gap between guess and measurement is where the learning is.

- During single-request decode on the T4, what utilisation will nvidia-smi show?
- What VRAM will the model hold at fp16? And at int8?
- Use week 2's formula for the VRAM. Guess the utilisation from today's trap slide.

Lab: profile the T4

Objective

Measure VRAM and utilisation across dtype and sequence length, and separate busy from well-used.

1

Baseline the T4 with nvidia-smi: total VRAM, what is already held, and why.

2

Load Qwen2.5-1.5B-Instruct at fp16. Record VRAM against your prediction card.

3

Reload at int8. Record VRAM, and note it holds fewer bytes but does not run faster.

4

Sweep sequence length. Watch memory-usage climb and log each point.

5

Run batch 1, then batch 8. Record tokens/sec and utilisation for each.

6

Compare: similar utilisation, very different tokens/sec. Write down why.



What you hand in

Deliverable

A profiling table across dtype and sequence length as `profile.json`, plus the batch 1 vs batch 8 result as `batch_check.json` - the verifier reads both.

Green check

The verifier reads `profile.json` and `batch_check.json` and confirms both are present and complete. An unattempted lab scores zero.

Recap

Memory first

Serving speed is set by bytes moved, not math done. The week runs on this.

The hierarchy

Weights live in device memory (GDDR6 on the T4, HBM on datacenter cards) and commute to the cores every token. That commute is the cost.

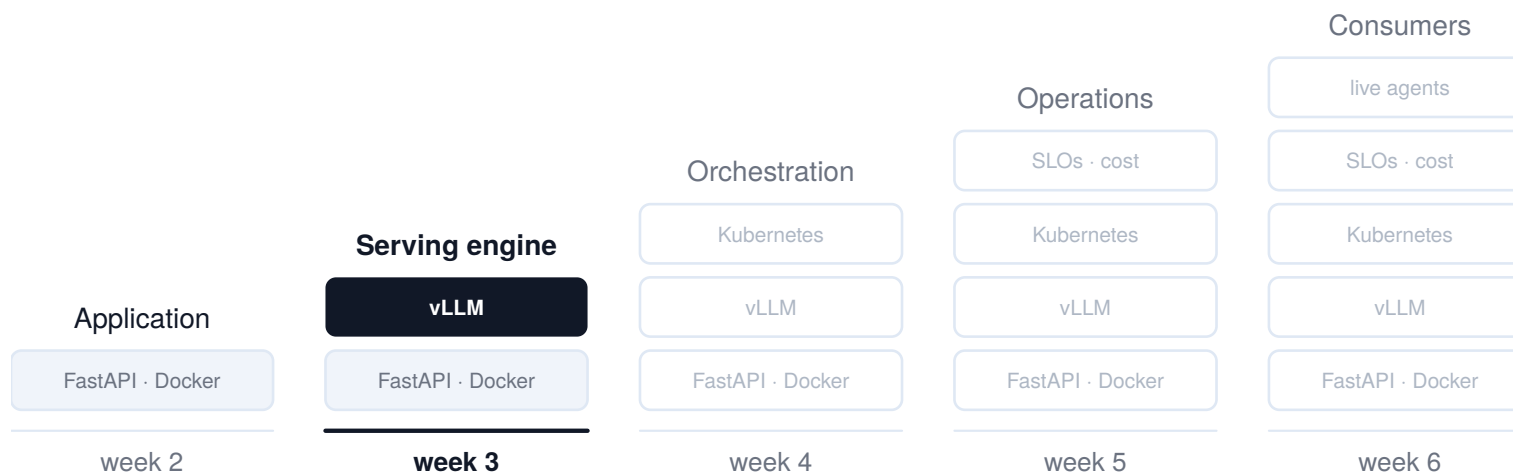
The util trap

Utilisation means a kernel ran, not that the card was used well.

Tomorrow: inside the loop

- Today we watched memory-usage climb during decode without saying why.
- Tomorrow we open prefill and decode properly, and compute the KV cache by hand.
- The amber region from week 2's tenant diagram becomes a number you can predict.

One system. A layer a week, and the stack is the course.



nvidia-smi field reference

Field	Reads	Use it to
Memory-Usage	used MiB / total MiB	check the next request fits; spot KV growth
GPU-Util	a kernel ran in the window	detect activity, not efficiency (the trap)
Pwr:Usage/Cap	watts drawn / cap	judge true work; a full card nears its cap
Temp / Perf	degrees C / P-state	catch thermal throttling
Processes	PID, type, memory held	see who holds the card when it is shared

These are the fields you read by hand. Week 5 scrapes the same values from DCGM into Prometheus.